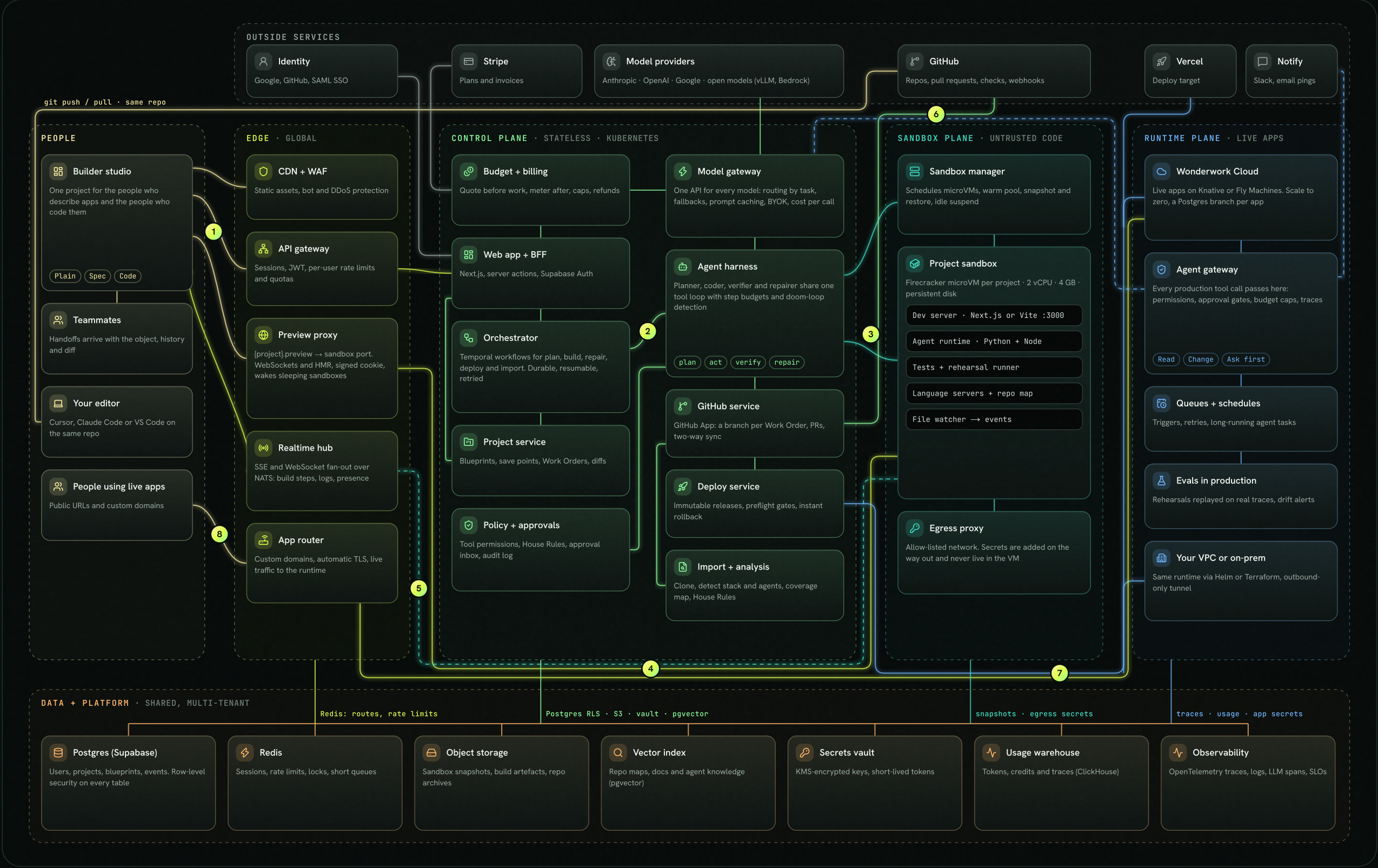


How Wonderwork runs *in production*.

Five planes with one job each. The control plane decides, sandboxes run untrusted code, the runtime serves live apps, and every model call and every agent action passes a gateway that can price it, cap it or ask a person first. The prototype you can click today implements the product surface of this design; the table at the end says exactly which parts are real.

- Diagram (PNG)
- Diagram + notes (PDF)
- Full write-up



THE EIGHT FLOWS ON THE DIAGRAM

- 1 Describe

The studio sends the brief through the API gateway. Nothing runs yet: the planner returns a Blueprint and a priced Work Order.
- 2 Plan

The orchestrator starts a durable workflow. The harness asks the model gateway, which picks a model per task and meters every call.
- 3 Build in a sandbox

Approved work runs inside the project's microVM: write files, install, run, test, rehearse agents. Every step is checkpointed.
- 4 Live preview

The preview proxy maps the project's subdomain to the sandbox dev server, including WebSockets for hot reload.
- 5 Show the work

File, test and step events stream back through the realtime hub, so people see progress in plain English, not a spinner.
- 6 Branch + PR

Each Work Order becomes a branch and a pull request through the GitHub App. Pushes from engineers sync back into the Blueprint.
- 7 Ship

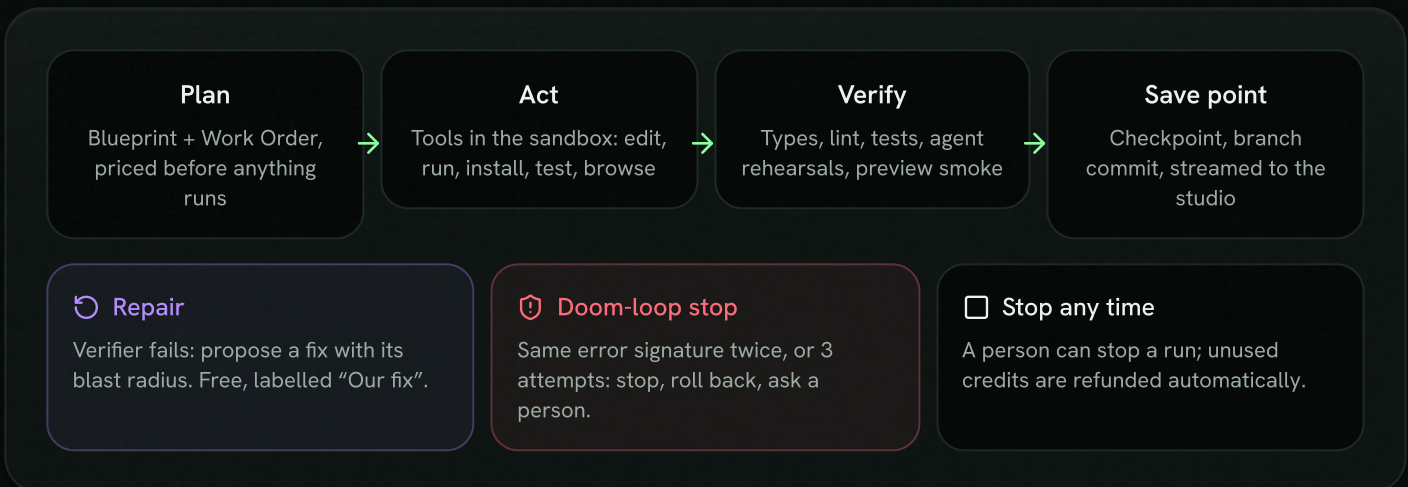
Preflight passes, the deploy service builds one immutable release and rolls it out to Wonderwork Cloud, Vercel or your VPC.
- 8 Governed agents

In production every tool call goes through the agent gateway: permissions, approval gates, budget caps and traces.

THE AGENT HARNESS

Plan, act, verify, repair. With a budget and a way out.

The model decides; deterministic code does the rest. The planner writes a Blueprint (structured JSON), code generation turns it into files, and a tool loop edits, runs and tests inside the sandbox. A verifier gates every step. When a check fails, the repairer tries a fix, but only within a step and cost budget. The same error twice means a doom loop: it stops, rolls back to the last save point and hands the problem to a person with the full context. Our own fixes are never billed.



SANDBOXING

One microVM per project, because the code is untrusted.

OPTION	VERDICT	ISOLATION	START	FIT FOR AGENTIC APPS
Firecracker microVMs	Chosen	Own kernel per project	~150 ms from snapshot	Runs Node and Python agents, real ports, persistent disk
gVisor containers	Fallback	User-space kernel	~1 s	Good isolation, some syscall and I/O overhead
Plain Docker	Rejected	Shared host kernel	~1 s	One escape reaches every tenant
Browser WebContainers	Rejected	Browser tab	Instant	Node only: no Python agents, no long-running jobs

SCALE

Thousands of people at once, without thousands of idle machines.

- 5,000
people in the studio at once
- ~1,500
sandboxes awake (30% active, the rest snapshotted)
- ~150 ms
resume from a warm snapshot
- ≤ 3
repair attempts before a person is asked

- ✓ The control plane is stateless; long work lives in durable workflows, so a pod restart never loses a build.
- ✓ The model gateway queues by priority per provider and key, fails over between providers, and caches prompts.
- ✓ Sandboxes suspend to a snapshot after 10 idle minutes and resume in about 150 ms from a warm pool.
- ✓ Events fan out over NATS; Postgres uses row-level security, read replicas and partitioned event tables.

HONEST ABOUT THE SEAMS

What the prototype runs today, and what production adds.

PART	IN THE LIVE PROTOTYPE	IN PRODUCTION
Studio, auth, data	Next.js 16 on Vercel, Supabase Auth (Google, email, guest sessions you can keep), Postgres with row-level security on every table	Same, plus SAML SSO, SCIM and regional data residency
Planner	Claude plans a structured Blueprint, streamed live; code expands it deterministically and validates every reference	Same contract, routed through the model gateway
Model gateway	One get(Model) seam, provider switch by env, per-call token and cost metering, daily budgets per person	Multi-provider routing, failover, BYOK, prompt caching
Agent gateway	Real approval gates in the playground: tools marked “Ask first” pause for a person before running	Every production tool call, with caps, audit and traces
Build + repair	Deterministic build timeline with a real repair decision that changes the Blueprint and creates a save point	Full tool loop inside microVMs with verifier and budgets
Sandbox + preview	Preview renders the Blueprint with a spec renderer inside the studio; no untrusted code runs	Firecracker microVM per project behind the preview proxy
GitHub	Reads any public repo, detects stack and agent frameworks, writes House Rules; pushes and PRs are sandboxed	GitHub App with branch per Work Order and two-way sync
Deploy	Wonderwork Cloud live URL is real (/live/...) with rollback; Vercel and VPC targets are sandboxed	Immutable releases, canary rollout, instant rollback